

面向对象编程 I — 思想

面向对象编程 · 2026 · 南京大学

| 刘钦

Outline

本讲是面向对象编程的第一讲（**思想层面**），回答三个核心问题：结构化编程为什么不够用？面向对象的核心思想是什么？如何用类和对象建模现实世界？

- **结构化编程的问题**

- 案例：SimpleDotComGame
- 可读性差 · 维护困难

- **面向对象思想**

- 从过程到职责
- 封装 · 抽象 · 多态

- **类和对象**

- 对象的概念
- 如何获得对象
- 类的定义与类图

- **案例分析**

- Real Sink Dot Com
- 职责分配与协作

结构化编程的问题

案例：SimpleDotComGame · 可读性与可维护性

用结构化方法我们怎么办？

自顶向下，逐步求精

Sink a Dot Com — 结构化设计

Game play begins

↓

Start a Game → Create Three Dotcom → Place them

↓

Command Data Board Data Dot Com Data Prompt

↓

Check the guess → Give a rating → Game Finishes

结构化方法把问题分解为一系列过程（函数），用全局数据在过程之间共享。

High-level Design 开发流程

一个类的开发方法

- 搞清楚类的**职责**（数据与行为）
- 列出**成员变量和方法**
- 写**伪代码**
- 写**测试用例**
- 实现这个类
- 测试
- 调试和修改 Bug

核心产出

阶段	输出
设计	职责清单
建模	成员变量 + 方法签名
规范	伪代码
验证	测试用例
实现	Java 代码
质保	全部测试通过

先想清楚"这个类要做什么"，再写代码。

写测试用例（TDD 思想）

```
public class SimpleDotComTestDrive {  
    public static void main(String[] args) {  
        SimpleDotCom dot = new SimpleDotCom();  
        int[] locations = {2, 3, 4};  
        dot.setLocationCells(locations);  
  
        String userGuess = "2";  
        String result = dot.checkYourself(userGuess);  
  
        String testResult = "failed";  
        if (result.equals("hit")) {  
            testResult = "passed";  
        }  
        System.out.println(testResult);  
    }  
}
```

先写测试，再写实现——测试代码就是对类行为的精确描述。

实现方法（第一版）

```
public class SimpleDotCom {
    int[] locationCells;
    int numOfHits = 0;

    public void setLocationCells(int[] locs) { locationCells = locs; }

    public String checkYourself(String stringGuess) {
        int guess = Integer.parseInt(stringGuess);
        String result = "miss";
        for (int cell : locationCells) {
            if (guess == cell) {
                result = "hit";
                numOfHits++;
                break;
            }
        }
        if (numOfHits == locationCells.length) {
            result = "kill";
        }
        return result;
    }
}
```

SimpleDotComGame — 主游戏循环

```
import java.util.Random;

class SimpleDotComGame {
    public static void main(String[] args) {
        int numOfGuesses = 0;
        GameHelper helper = new GameHelper();
        SimpleDotCom theDotCom = new SimpleDotCom();
        int randomNum = new Random().nextInt(4);
        theDotCom.setLocationCells(
            new int[]{randomNum, randomNum+1, randomNum+2});

        boolean isAlive = true;
        while (isAlive) {
            String guess = helper.getUserInput("enter a number");
            String result = theDotCom.checkYourself(guess);
            numOfGuesses++;
            if (result.equals("kill")) {
                isAlive = false;
                System.out.println("You took " + numOfGuesses + " guesses");
            }
        }
    }
}
```

修 Bug —— 发现判断漏洞

有漏洞的版本

```
for (int cell : locationCells) {  
    if (guess == cell) {  
        result = "hit";  
        numOfHits++;  
        break;  
    }  
}  
if (numOfHits == locationCells.length) {  
    result = "kill";  
}
```

问题： 同一个位置被猜多次时，`numOfHits` 会重复累加——判断"kill"的条件不可靠！

修复版：用 ArrayList

```
public String checkYourself(String userInput) {  
    String result = "miss";  
    int index = locationCells.indexOf(userInput);  
    if (index >= 0) {  
        locationCells.remove(index); // ★ 猜中就移除  
        if (locationCells.isEmpty()) {  
            result = "kill";  
        } else {  
            result = "hit";  
        }  
    }  
    return result;  
}
```

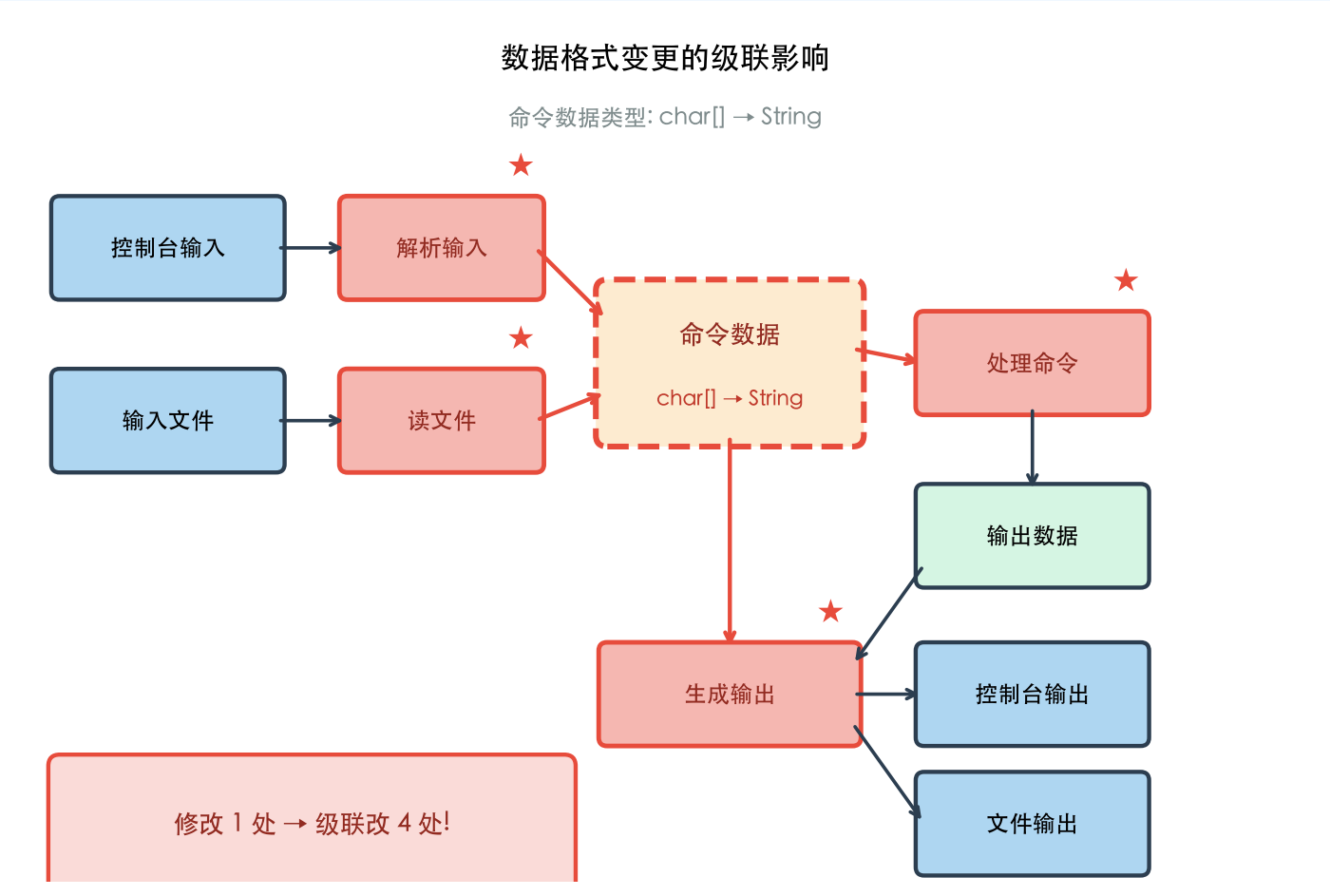
用 `ArrayList` 代替 `int[]`，猜中即移除——天然去重。

结构化编程的根本问题

问题	表现	后果
Not Easy to Read	全局变量到处被访问修改	第 11 行改的数据在第 137 行读取
Not Easy to Maintain	实现变更牵一发动全身	<code>char[]</code> 变 <code>String</code> 导致多处修改
Hard to Extend	新增需求要改原有代码	多种输出方式？大范围修改

根源：数据与行为分离——数据暴露在全局，谁都能改；行为分散在多个函数，修改一处波及多处。

可维护性问题 — 实现变更

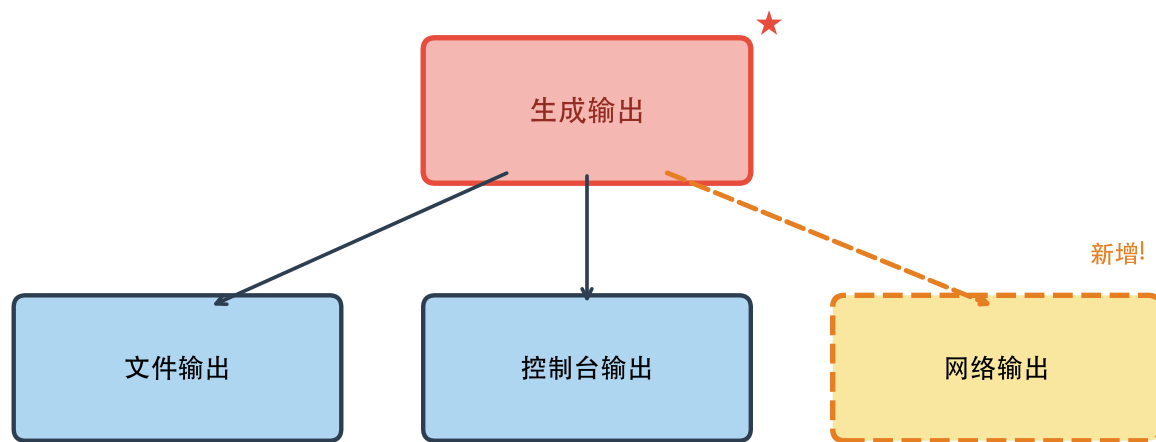


数据格式的改变会传播到所有读取这个数据的过程——大范围的修改 Nightmare!

可维护性问题 — 需求 A：新增一种输出方式

需求 A：新增一种输出方式

已有文件输出和控制台输出，现在要加第三种：网络输出



问题：新增输出方式 → 必须修改 "生成输出" 的 if-else 分支

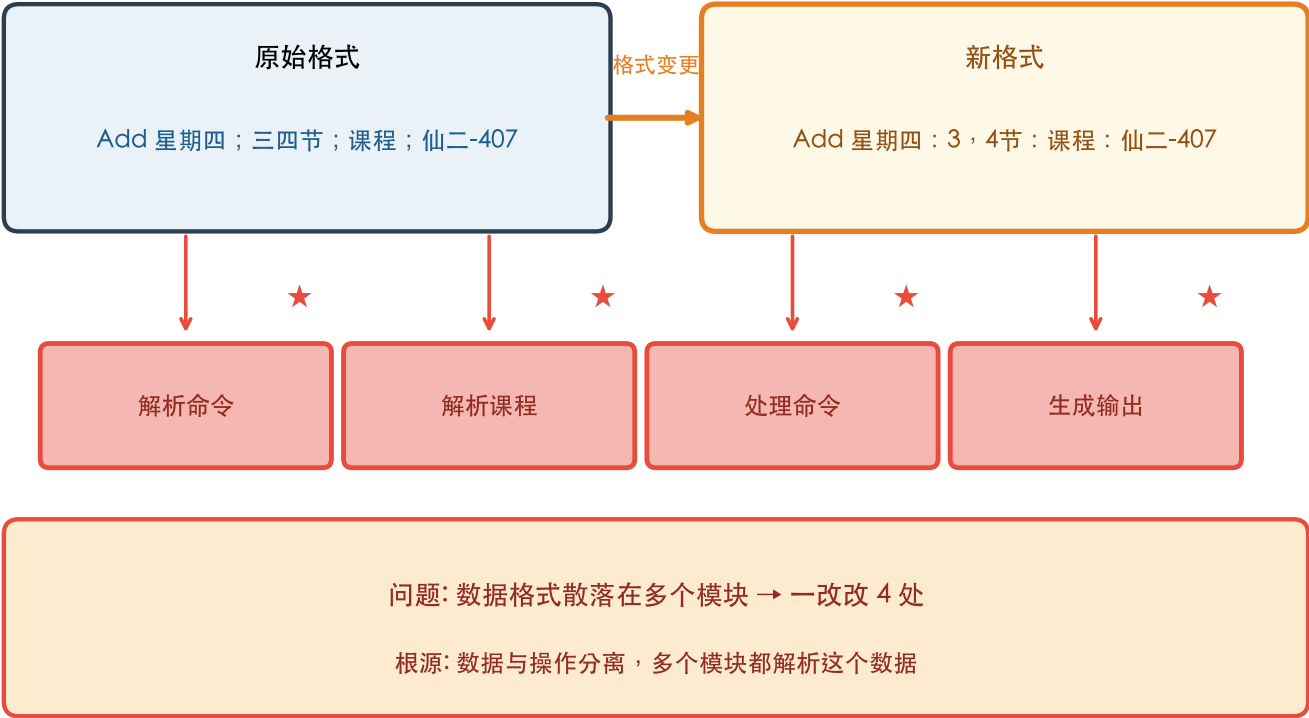
每变一次，核心模块就要改一次

问题本质： 输出行为被写死在 "生成输出" 模块的 if-else 分支里——每增加一种输出方式，都要修改这个核心模块。

可维护性问题 — 需求 B：输出格式发生变更

需求 B: 输出格式变更

课程数据字段分隔符从 “;” 改为 “:”



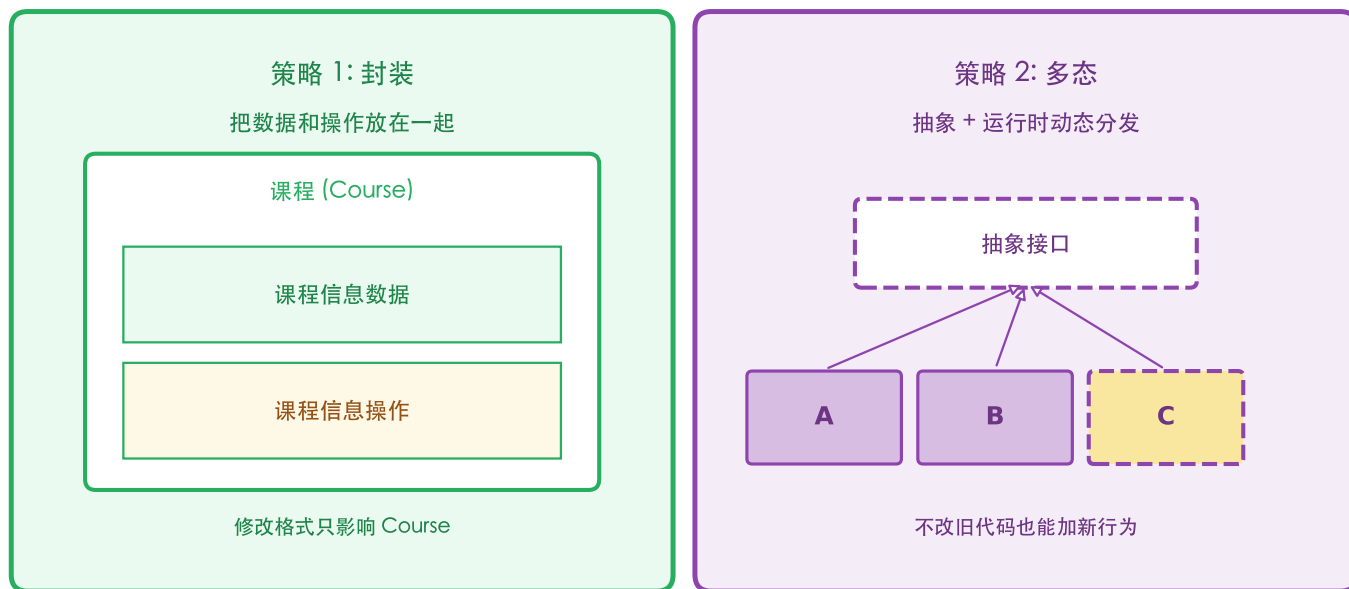
问题本质: 数据格式散落在多个模块——解析命令、解析课程、处理命令、生成输出都独立地解析和格式化同一份数据。改一次要改 4 处。

Q: 不修改，怎么应对变更？

A:

1. 在有限的范围内修改
2. 扩展（不动旧代码，加新代码）

两个关键突破口



有限范围修改 — 数据与操作在一起（封装）

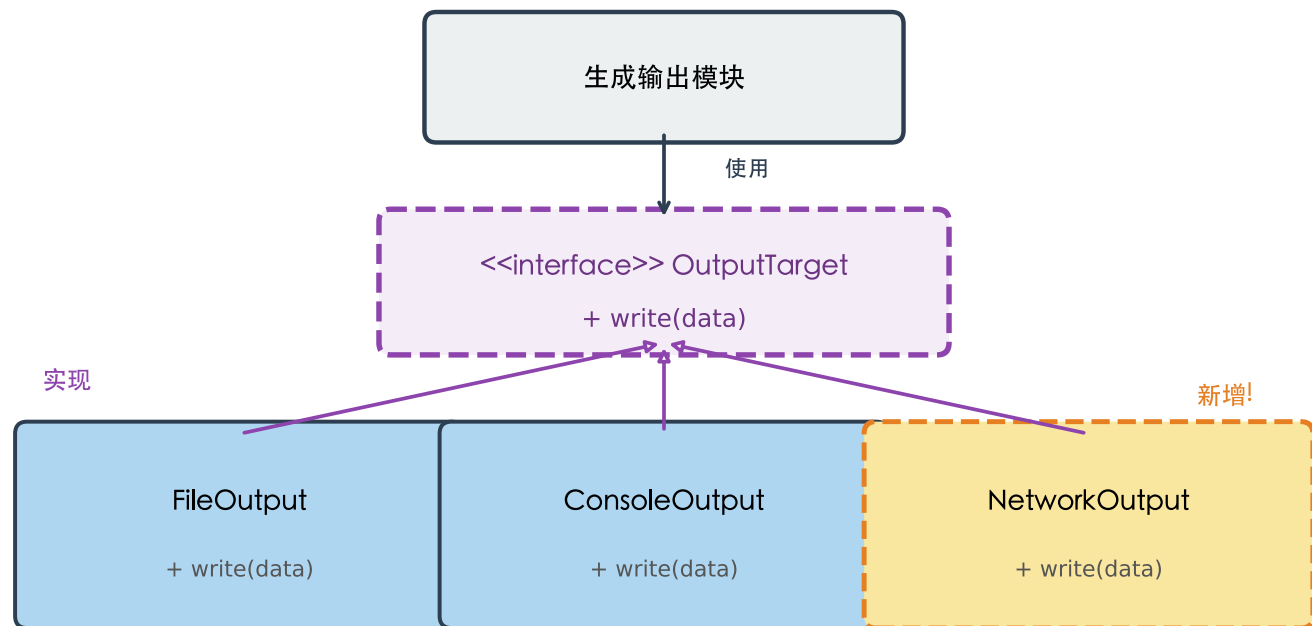
Course 封装示意



封装：数据被隐藏在对象内部，外部只能通过**公开接口**操作——这就是类的本质。针对需求 B（格式变更），只需改 Course 内部的 `parse()` 和 `format()` 两个方法，**外部调用者完全不受影响**。

扩展 — 运行时动态链接（多态预告）

多态: 定义抽象接口, 运行时动态分发



抽象 + 多态：定义统一的输出接口，运行时根据具体类型动态分发——针对需求 A（新增输出方式），只需加新类实现接口，旧代码一行都不改。

面向对象思想

职责 · 数据 · 行为 · 封装

视角的变化

视角	结构化方法	面向对象方法
数据视角	数据为中心	职责视角
行为视角	方法的集合	职责的实现
思维方式	函数之间的调用	有职责的对象之间的交互

你的世界观改变了！
以前是函数之间的调用，现在是有职责的对象之间的交互。

职责 (Responsibility)

定义

职责 = 数据职责 + 行为职责

一个类的数据和行为应当能体现一个职责。

练习：这是一个职责吗？

```
class Person {  
    String name;  
    int getAge() { ... }  
}
```

✗ 不是——name 和 getAge() 没有关联：age 从哪来？

职责 — 数据和行为在一起

反例 1: 数据与行为不配对

```
class Person {  
    String name;  
    int getAge() { ... } // age 从哪来?  
}
```

反例 2: 行为缺少数据支撑

```
class Person {  
    Date birthday;  
    int getAge() { ... }  
}
```

行为单一但过于贫血。

正例: 数据和行为相互支撑

```
class Person {  
    String name;  
    Date birthday;  
  
    public String getName() { return name; }  
    public int getAge() {  
        return computeAge(birthday);  
    }  
}
```

- `name` $\leftarrow \rightarrow$ `getName()`
- `birthday` $\leftarrow \rightarrow$ `getAge()`

职责内聚: 数据和行为围绕同一个概念。

再看课程表应用中的职责

数据职责

数据	说明
命令数据	命令的文本/结构
课程数据	课程信息
文件数据	文件路径/内容

行为职责

行为	属于谁?
命令的解析	命令
处理命令	课程表
课程数据的解析	课程
控制台输入	控制台
控制台输出	控制台
文件输入	文件
文件输出	文件
生成输出	课程表

找到数据 + 行为的配对，就找到了类。

职责的分配 — 类的诞生

职责分配 — 类的诞生



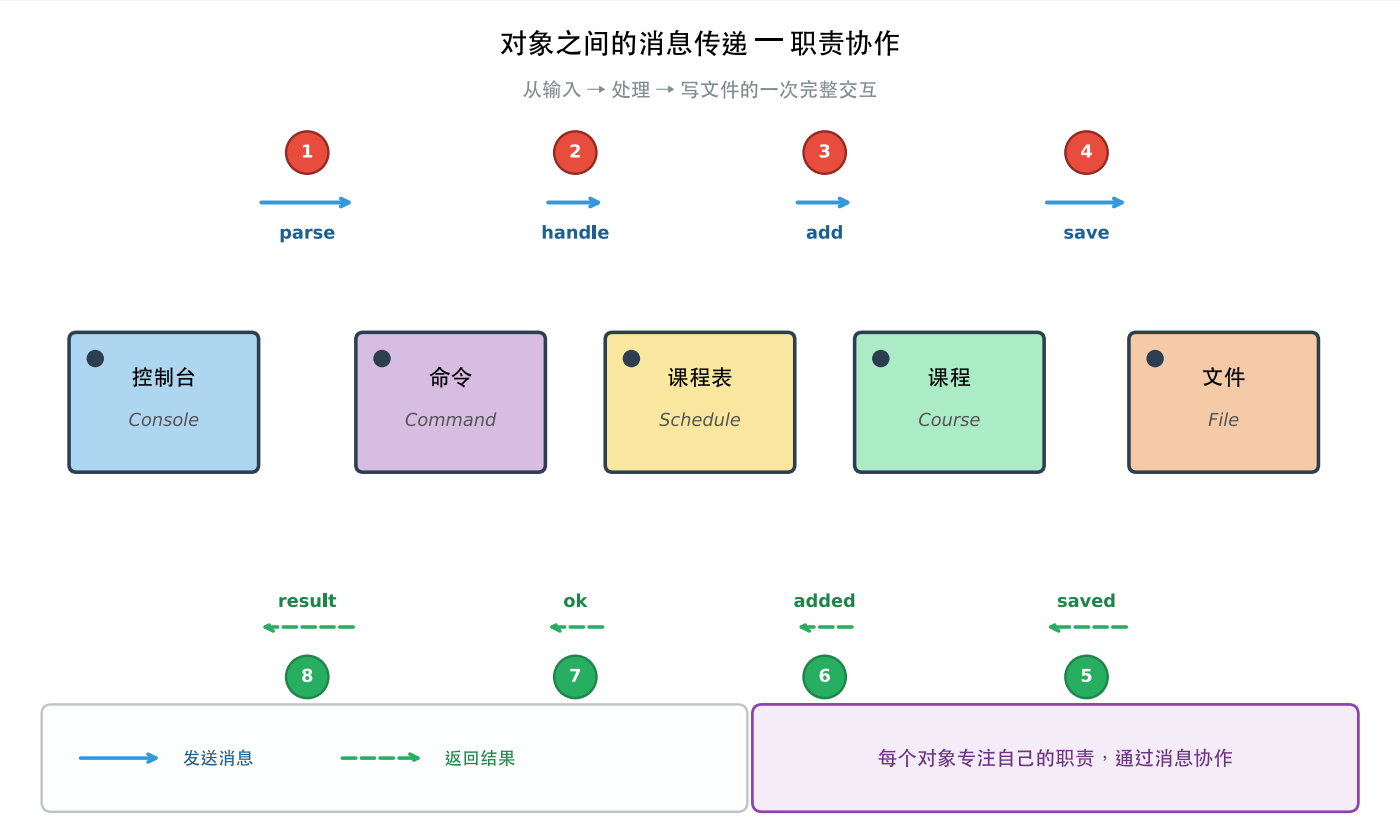
↓ 从数据和行为找对应的类

原始需求中的联合职责

命令数据· 课程数据· 文件数据· 命令解析· 处理命令· 生成输出· 读写

数据和行为配对 → 4 个类

类 — 职责的抽象（对象之间的消息传递）



类是职责的抽象，对象是职责的实现——他们之间通过消息传递（方法调用）协作完成复杂任务。

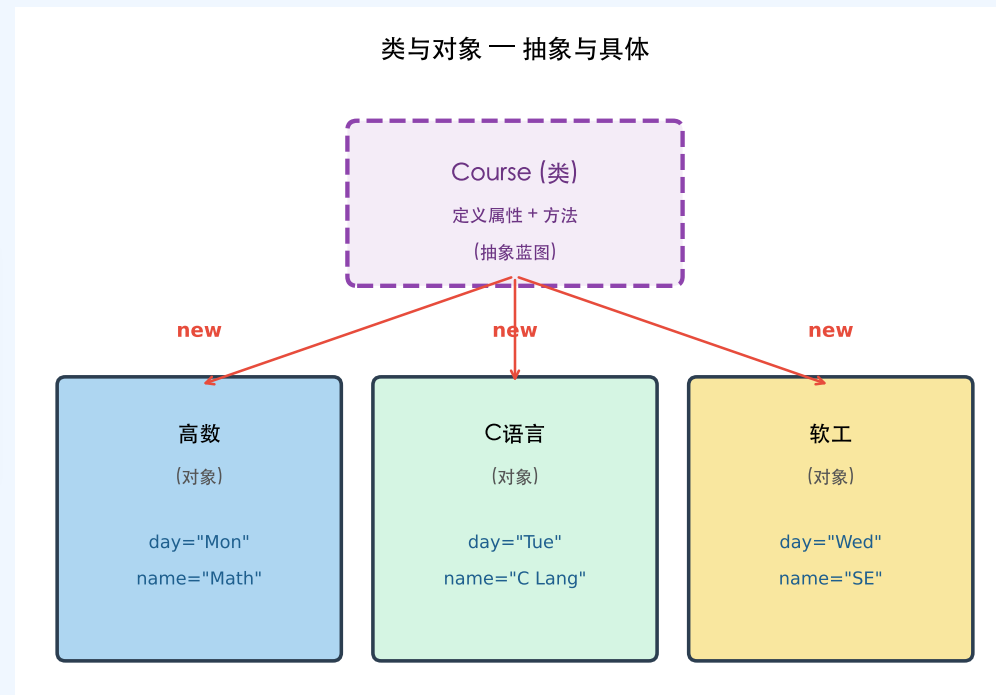
对象 — 职责的实现

课程只有一个？

不，课程有很多个！

课程 1：高数
课程 2：C 语言
课程 3：软件工程
...

每个课程都是一个**独立的对象**，它们的**行为代码共享**（都来自 Course 类），但**数据各自独立**。



类是模板，对象是实例。
类与对象的关系 = 抽象与具体。

类和对象

对象的本质 · 如何找对象 · 类的定义

什么是对象？

对象是面向对象方法中的术语，既表示客观世界问题空间中的某个具体的事物，又表示软件系统解空间中的基本元素。

对象的三要素

要素	说明
唯一标识符 (Identity)	每个对象的身份永远不同
属性 (Properties)	需要记忆的信息（状态）
方法 (Methods)	对象能够提供的服务（行为）

对象的三条规则

规则 1: 记忆

每个对象都保存着描述当前特征的信息。

```
Dog d = new Dog();  
d.name = "Rex";  
d.age = 3;  
// d 记住了自己的 name 和 age
```

规则 2: 状态变化

对象状态的改变必须通过调用方法实现。

```
// ❌ 直接改字段 (破坏封装)  
d.age = 4;  
// ✅ 调用方法  
d.birthday(); // 内部做 age++
```

规则 3: 唯一性

每个对象的标识永远是不同的，状态常常也存在着差异。

```
Dog d1 = new Dog("Rex");  
Dog d2 = new Dog("Rex");  
// d1 和 d2 同名同年龄  
// 但它们是两个不同的对象!  
d1 == d2 // false (不同一)  
d1.equals(d2) // 需要重写才返回 true
```

回顾 Lab06: 这就是 `==` vs `.equals()` 的本质——同一性 vs 等价性。

如何获得对象？

寻找候选对象

- 找名词 → 类（对象）与属性
- 找动词 → 行为

精化对象

- 去除冗余 — 重复的概念合并
- 去除不相干 — 与问题无关的概念
- 去除模糊 — 不明确的概念
- 转化 — 没有行为的对象 → 某个类的属性

例子：课程表应用

名词候选：

- 命令、课程、课程表
- 输入、输出
- 文件、控制台、数据格式
- 用户（→ 超出范围，去除）

动词候选：

- 解析命令、解析课程
- 处理命令、生成输出
- 读取输入、写入输出

经验：需求文档中的名词/动词是找对象的起点。

什么是类？

面向对象程序设计语言使用**类**来描述对象，并且通过类方法来定义它们的行为。

核心概念

- **类这个术语**被用来描述相同事物的集合
- 类以概要的方式描述了相同事物集合中的**所有元素**
- 但允许类中的每一个**实体元素**可以在非本质特征上变化

编程中的类 = 现实中的分类

编程中使用类的思想与现实世界中**把东西进行分类**的思想一致——这是一种方便、明确的事物组织方式。

类与对象 — 创建过程

类的本质

- 类是一个描述或**蓝图**（被表示成**一段代码**），用于定义组成某类特定对象的**所有特性**
- 一旦定义了一个类，就可以得到这个类的**对象或实例**

```
public class Person {  
    String name; // 实例变量  
    int age;  
  
    public void greet() { ... }  
}
```

对象的创建

- 当创建类的实例时，就建立了这种类型的一个**对象**
- 系统为**类定义的实例变量分配内存**
- 实例变量的值**由类的每个实例提供**

```
Person alice = new Person();  
alice.name = "Alice"; // 为实例变量赋值  
  
Person bob = new Person();  
bob.name = "Bob";  
// alice 和 bob 有独立的 name 内存
```

职责 — 单一职责原则 (SRP)

每个类应当只有单一的职责。

判断标准

当你发现有两个变化会要求我们修改这个类，那么你就要考虑拆分这个类了。

给对象分配责任的策略

- 寻找需要执行的动作（行为）
- 以及需要维护和生成的信息（数据）
- 覆盖到所有重要的面

单一职责是 OOP 的核心原则之一，也是良好设计的第一步。

创建类的原因

原因	说明
对现实世界建模	Person, Dog, Course ← 有形物体
对抽象对象建模	Event, Process, Transaction ← 无形概念
降低复杂度	把大问题分成小对象
隔离复杂度	复杂的算法封装在一个类里
隐藏实现细节	客户只看接口，不看实现
限制变化影响范围	修改局部化
创建中心控制点	配置、单例等

一个类存在的理由，决定了它的设计质量。

面向对象分析方法

方法 1: 用例分析

- 从用户使用场景出发
- 每个用例描述一个完整的任务流程
- 从用例中抽取对象

方法 2: CRC 卡

- Class — 类名
- Responsibility — 职责
- Collaborator — 协作者
- 一张卡片一个类，快速头脑风暴

方法 3: 非正式英语描述

- 用自然语言描述需求
- 名词找类，动词找方法

类的定义 — 语法结构

```
class MyClass {  
    // 字段 (field) - 数据  
    private int x;  
  
    // 构造方法 (constructor) - 对象初始化  
    public MyClass(int initX) {  
        this.x = initX;  
    }  
  
    // 方法 (method) - 行为  
    public int getX() {  
        return x;  
    }  
}
```

类的三大组成部分：字段 + 构造方法 + 方法。

类的定义 — Bicycle 完整示例

```
public class Bicycle {
    // 字段（实例变量）
    private int cadence;
    private int gear;
    private int speed;

    // 构造方法
    public Bicycle(int startCadence,
                   int startSpeed,
                   int startGear) {
        cadence = startCadence;
        speed = startSpeed;
        gear = startGear;
    }

    // Getter
    public int getCadence() { return cadence; }
    public int getSpeed() { return speed; }
```

```
// Setter
public void setCadence(int newValue) {
    cadence = newValue;
}

// 行为方法
public void applyBrake(int decrement) {
    speed -= decrement;
}
public void speedUp(int increment) {
    speed += increment;
}
}
```

类型	作用
cadence/gear/speed	字段（状态）
Bicycle(...)	构造方法
applyBrake/...	方法（行为）

类的定义 — Point 简化示例

```
public class Point {  
    public int x = 0;  
    public int y = 0;  
  
    // constructor  
    public Point(int a, int b) {  
        x = a;  
        y = b;  
    }  
}  
  
// 使用  
Point originOne = new Point(23, 94);
```

`new Point(23, 94)` 过程:

1. JVM 在堆上分配 Point 对象的内存
2. 调用构造方法 `Point(23, 94)`
3. 字段 `x=23` , `y=94`
4. 返回该对象的引用, 赋给 `originOne`

变量和方法的访问

字段访问

```
// 方式 1: 访问匿名对象的字段（一次性用）  
int height = new Rectangle().height;  
  
// 方式 2: 通过引用变量访问  
Rectangle rect = new Rectangle();  
int height = rect.height;
```

方法调用

```
// 语法: 引用变量.方法名(参数)  
System.out.println("Area: " + rectOne.getArea());  
rectTwo.move(40, 72);
```

使用 `.` 操作符访问对象的字段和方法——这是面向对象的点号表示法（dot notation）。

案例分析

A Real "Sink a Dot Com" — 完整 OO 设计

A Real "Sink a Dot Com"

游戏规则

- **Goal:** Sink all Dot Coms in the fewest number of guesses
- **Setup:** Three Dot Coms on **7x7 grid**
- **How to play:**
 - Guess → Hit, miss or kill
 - Guess → Hit, miss or kill
 - ...
 - Win, show your number of guesses

相比 Simple 版的变化

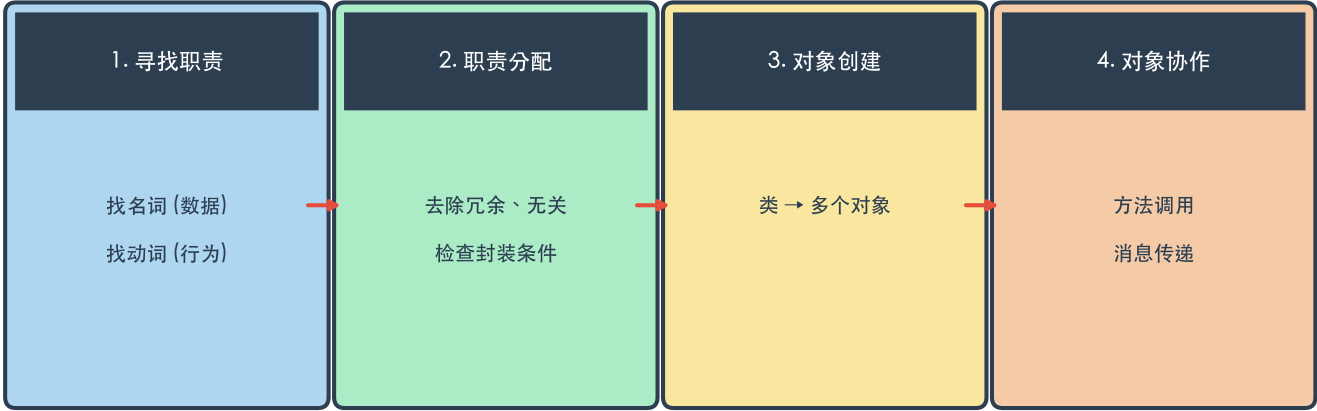
方面	Simple	Real
Dot Com 数量	1	3
位置	1D	2D
算法	Easy	Complicated

用结构化方法？复杂度爆炸。

用面向对象方法？职责分配，协作。

面向对象的开发过程

面向对象的开发过程



寻找名词与动词

名词（候选数据）

- 游戏
- 棋盘
- 猜测
- 提示
- Dot Com
- 公司数
- 公司名字
- 公司位置
- 目标

动词（候选行为）

- 游戏开始
- 游戏结束
- 显示猜测次数
- 判断游戏是否结束
- 判断是否猜中
- 输入
- 输出

从需求描述中逐字逐句地找——这是 OO 分析的第一步。

职责分配 — 两个核心类

DotComBust（游戏总控）

数据职责

- DotCom 集合
- 猜测次数
- IOHelper（输入辅助）

行为职责

- 游戏开始
- 判断游戏结束
- 游戏结束（显示猜测次数）

对象个数：1 个（唯一的实例）

DotCom（单个公司）

数据职责

- DotCom 名字
- DotCom 位置

行为职责

- 判断是否猜中

对象个数：3 个（3 家公司）

类的职责 vs 对象的个数——类描述共性，对象描述实例。

DotComBust — 类的骨架

```
import java.util.*;

public class DotComBust {
    private GameHelper helper = new GameHelper();
    private ArrayList<DotCom> dotComsList = new ArrayList<>();
    private int numOfGuesses = 0;

    private void setUpGame() { ... }
    private void startPlaying() { ... }
    private void checkUserGuess(String userGuess) { ... }
    private void finishGame() { ... }

    public static void main(String[] args) {
        DotComBust game = new DotComBust();
        game.setUpGame();
        game.startPlaying();
    }
}
```

类的骨架先确定——字段 + 方法签名，稍后填充实现。

DotComBust — setUpGame

```
private void setUpGame() {
    DotCom one = new DotCom();
    one.setName("Pets.com");
    DotCom two = new DotCom();
    two.setName("eToys.com");
    DotCom three = new DotCom();
    three.setName("Go2.com");
    dotComsList.add(one);
    dotComsList.add(two);
    dotComsList.add(three);

    System.out.println("Your goal is to sink three dot coms.");
    System.out.println("Pets.com, eToys.com, Go2.com");
    System.out.println("Try to sink them all in the fewest number of guesses");

    for (DotCom dotComSet : dotComsList) {
        ArrayList<String> newLocation = helper.placeDotCom(3);
        dotComSet.setLocationCells(newLocation);
    }
}
```

DotComBust — 主循环与检查

startPlaying / finishGame

```
private void startPlaying() {
    while (!dotComsList.isEmpty()) {
        String g = helper.getUserInput("guess");
        checkUserGuess(g);
    }
    finishGame();
}

private void finishGame() {
    System.out.println("All Dot Coms dead!");
    if (numOfGuesses <= 18)
        System.out.println("只用了 " +
            numOfGuesses + " 次猜测");
    else
        System.out.println("猜了 " +
            numOfGuesses + " 次, 太多了");
}
```

checkUserGuess — 核心逻辑

```
private void checkUserGuess(String userGuess) {
    numOfGuesses++;
    String result = "miss";
    for (DotCom d : dotComsList) {
        result = d.checkYourself(userGuess);
        if (result.equals("hit")) break;
        if (result.equals("kill")) {
            dotComsList.remove(d);
            break;
        }
    }
    System.out.println(result);
}
```

委托：DotComBust 不直接判断是否命中，而是委托给每个 DotCom 自己判断。

DotCom — 单个公司类

```
import java.util.ArrayList;

public class DotCom {
    private ArrayList<String> locationCells;
    private String name;

    public void setLocationCells(ArrayList<String> loc) { locationCells = loc; }
    public void setName(String string) { name = string; }

    public String checkYourself(String userInput) {
        String result = "miss";
        int index = locationCells.indexOf(userInput);
        if (index >= 0) {
            locationCells.remove(index);
            if (locationCells.isEmpty()) {
                result = "kill";
            } else {
                result = "hit";
            }
        }
        return result;
    }
}
```

GameHelper — 辅助工具类 (1/2)

```
import java.io.*;
import java.util.*;

public class GameHelper {
    private static final String alphabet = "abcdefg";
    private int gridLength = 7;
    private int gridSize = 49;
    private int[] grid = new int[gridSize];
    private int comCount = 0;

    public String getUserInput(String prompt) {
        String inputLine = null;
        System.out.print(prompt + " ");
        try {
            BufferedReader is = new BufferedReader(
                new InputStreamReader(System.in));
            inputLine = is.readLine();
            if (inputLine.length() == 0) return null;
        } catch (IOException e) {
            System.out.println("IOException: " + e);
        }
        return inputLine.toLowerCase();
    }
}
```

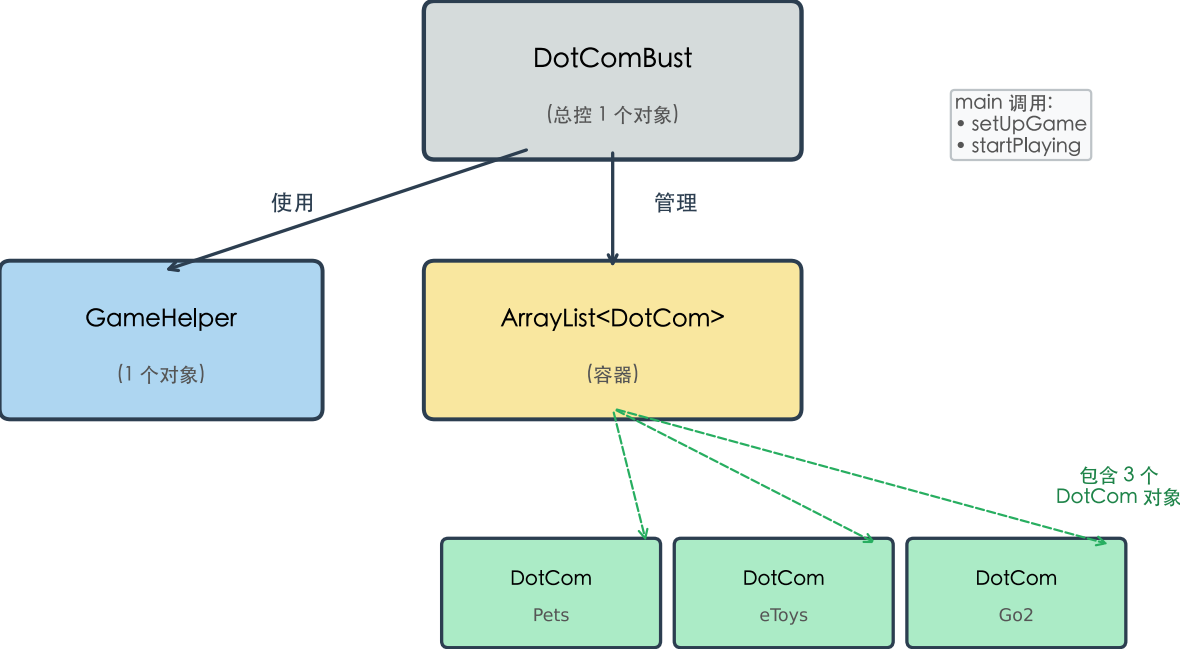
GameHelper — placeDotCom (2/2)

```
public ArrayList<String> placeDotCom(int comSize) {
    ArrayList<String> alphaCells = new ArrayList<>();
    int[] coords = new int[comSize];
    boolean success = false;
    int attempts = 0;
    comCount++;
    int incr = (comCount % 2 == 1) ? gridLength : 1; // 垂直/水平放置

    while (!success && attempts++ < 200) {
        int location = (int)(Math.random() * gridSize);
        int x = 0;
        success = true;
        while (success && x < comSize) {
            if (grid[location] == 0) {
                coords[x++] = location;
                location += incr;
                if (location >= gridSize) success = false;
                if (x > 0 && (location % gridLength == 0)) success = false;
            } else success = false;
        }
    }
    // ... 转换为 alpha 坐标 (如 "a3") 并返回
    return alphaCells;
}
```

类之间的协作关系

DotComBust 类之间的协作关系



交互链

```
User → DotComBust.startPlaying()  
      → GameHelper.getUserInput()  
      → checkUserGuess()
```

OO vs 结构化 — 本质对比

维度	结构化	面向对象
组织单位	函数	类 + 对象
数据	全局共享	封装在对象内
行为	独立过程	对象的方法
耦合度	高（数据-过程紧耦合）	低（接口隔离）
可扩展	修改原代码	扩展新类
可测试	难（全局状态）	易（对象独立）
世界观	流程	协作

结构化是以任务为中心，面向对象是以对象为中心——这是思维方式的根本变化。

本讲总结

面向对象编程 I – 思想

- 结构化编程的问题
 - 全局数据 · 难读难维护 · 扩展困难
- 面向对象的核心思想
 - 职责 = 数据 + 行为（封装）
 - 类是职责的抽象，对象是职责的实现
- 类和对象
 - 对象 = 标识 + 属性 + 方法
 - 类 = 蓝图，对象 = 实例
 - 单一职责原则（SRP）
- OO 开发过程
 - 寻找职责 → 职责分配 → 对象创建 → 对象协作
 - 案例：Real Sink Dot Com (DotComBust + DotCom + GameHelper)

- 下一讲：面向对象编程 II — 继承（类的层次结构与代码复用）